

Lab 6 – Đường đi ngắn nhất & MST

1. Mục tiêu

Kết thúc Lab 6, sinh viên có thể:

- Cài đặt và chạy được Dijkstra với heapq để tìm đường đi ngắn nhất.
- Đọc, in và giải thích bảng khoảng cách $d[v]$ và mảng parent.
- Cài đặt Union-Find (DSU) cơ bản, sau đó nâng cấp với path compression + union by size.
- Áp dụng DSU để cài đặt Kruskal tìm MST.
- Hiểu ở mức trực quan vì sao độ phức tạp chủ yếu là sort cạnh + thao tác DSU.

2. Cấu trúc Lab

Bài 1 – Dijkstra và bảng $d[v]$

File: lab6_bai1.py

Biểu diễn đồ thị

Sử dụng dict: mỗi đỉnh trỏ tới list (neighbor, weight):

```
graph = {
    'A': [('B', 4), ('D', 1)],
    'B': [('A', 4), ('C', 2), ('E', 3)],
    'C': [('B', 2), ('F', 5)],
    'D': [('A', 1), ('E', 2)],
    'E': [('D', 2), ('B', 3), ('F', 1)],
    'F': [('E', 1), ('C', 5)]
}
```

Code Dijkstra

```
import heapq

def dijkstra(graph, source):
    """
    Dijkstra: tìm đường đi ngắn nhất từ source đến tất cả các đỉnh.
    graph: dict {vertex: [(neighbor, weight), ...]}
    source: đỉnh bắt đầu
    """

    # Bước 1: khởi tạo khoảng cách
    distances = {v: float('inf') for v in graph}
    distances[source] = 0

    # Lưu cha để reconstruct path
    parent = {v: None for v in graph}

    # Priority queue: (khoảng_cách_tạm_thời, đỉnh)
    pq = [(0, source)]

    # Để không xử lý 1 đỉnh 2 lần
    visited = set()

    while pq:
        # Lấy đỉnh có khoảng cách hiện tại nhỏ nhất
        current_dist, u = heapq.heappop(pq)

        # Nếu đã xử lý u rồi thì bỏ qua
        if u in visited:
            continue
```

```

visited.add(u)

# Nếu khoảng cách trong heap cũ hơn distances[u], bỏ qua
if current_dist > distances[u]:
    continue

# Thử đi qua u để “nói lòng” các hàng xóm
for v, w in graph[u]:
    new_dist = distances[u] + w
    if new_dist < distances[v]:
        distances[v] = new_dist
        parent[v] = u
        heapq.heappush(pq, (new_dist, v))

return distances, parent

```

- `distances[v]`: “tốt nhất hiện giờ” cho đoạn `source` $\rightarrow$ `v`.
- `parent[v]`: trên đường đi ngắn nhất, trước `v` là ai.
- `heapq`: luôn lôi ra đỉnh có khoảng cách hiện tại nhỏ nhất.

Hàm dựng đường đi từ parent

```

def reconstruct_path(parent, source, target):
    """
    Dựng lại đường đi từ source đến target dùng mảng parent.
    """
    path = []
    cur = target
    while cur is not None:
        path.append(cur)

```

```

    cur = parent[cur]
path.reverse()
# Nếu đỉnh đầu không phải source → không có đường đi
if not path or path[0] != source:
    return None
return path

```

Hàm test và in bảng d[v]

```

def print_distances(distances, source):
    print(f"Bảng khoảng cách từ {source}:")
    for v in sorted(distances.keys()):
        d = distances[v]
        if d == float('inf'):
            print(f" {source} → {v}: INF (không tới được)")
        else:
            print(f" {source} → {v}: {d}")

def test_dijkstra():
    graph = {
        'A': [('B', 4), ('D', 1)],
        'B': [('A', 4), ('C', 2), ('E', 3)],
        'C': [('B', 2), ('F', 5)],
        'D': [('A', 1), ('E', 2)],
        'E': [('D', 2), ('B', 3), ('F', 1)],
        'F': [('E', 1), ('C', 5)]
    }

    source = 'A'
    distances, parent = dijkstra(graph, source)

```

```

print_distances(distances, source)
print("\nĐường đi chi tiết:")
for v in sorted(graph.keys()):
    if v == source:
        continue
    path = reconstruct_path(parent, source, v)
    if path is None:
        print(f" {source} → {v}: không có đường đi")
    else:
        cost = distances[v]
        print(f" {source} → {v}: {' → '.join(path)} (cost = {cost})")

if __name__ == "__main__":
    test_dijkstra()

```

Yêu cầu SV: chạy, rồi vẽ lại đồ thị, tự kiểm tra $d[v]$ và đường đi.

Bài 2 – Union-Find (DSU)

File: lab6_bai2.py

DSU phiên bản 1

```

def make_set(vertices):
    """
    Khởi tạo DSU: mỗi đỉnh là một nhóm riêng, parent[v] = v.
    """
    parent = {}
    for v in vertices:
        parent[v] = v
    return parent

```

```
def find(parent, v):
    """
    Tìm root (trưởng nhóm) của v, leo lên tới khi parent[v] == v.
    """
    while parent[v] != v:
        v = parent[v]
    return v
```

```
def union(parent, a, b):
    """
    Gộp nhóm chứa a và nhóm chứa b.
    """
    root_a = find(parent, a)
    root_b = find(parent, b)
    if root_a != root_b:
        parent[root_b] = root_a # cho root_b nhập hội root_a
```

```
def demo_dsu_basic():
    vertices = ['A', 'B', 'C', 'D', 'E']
    parent = make_set(vertices)
```

```
    ops = [
        ("union", 'A', 'B'),
        ("union", 'C', 'D'),
        ("find", 'B'),
        ("union", 'B', 'C'),
        ("find", 'D'),
        ("find", 'E'),
    ]
```

```

for op in ops:
    if op[0] == "union":
        _, x, y = op
        print(f"Thực hiện union({x}, {y})")
        union(parent, x, y)
    else:
        _, x = op
        root = find(parent, x)
        print(f"find({x}) = {root}")

print(" parent hiện tại:", parent)

```

Yêu cầu SV: chạy, đọc log, vẽ “cây nhóm” tương ứng trên giấy.

DSU phiên bản 2

```

def make_set_optimized(vertices):
    """
    Khởi tạo DSU với size để union by size.
    """
    parent = {}
    size = {}
    for v in vertices:
        parent[v] = v
        size[v] = 1
    return parent, size

def find_optimized(parent, v):
    """
    Path compression: nén đường đi về root.

```

```

"""
if parent[v] != v:
    parent[v] = find_optimized(parent, parent[v])
return parent[v]

def union_optimized(parent, size, a, b):
    """
    Union by size: gắn cây nhỏ vào cây lớn.
    """
    root_a = find_optimized(parent, a)
    root_b = find_optimized(parent, b)
    if root_a == root_b:
        return

    # root_a là cây lớn hơn
    if size[root_a] < size[root_b]:
        root_a, root_b = root_b, root_a

    parent[root_b] = root_a
    size[root_a] += size[root_b]

```

Nhiệm vụ cho SV:

- So sánh `parent` trước và sau khi gọi nhiều lần `find_optimized`.

So sánh basic vs optimized

- Tạo chuỗi dài `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)`.
- Gọi `find` nhiều lần, in số bước (đếm vòng `while`) của bản basic vs optimized.

Bài 3 – Kruskal MST với DSU

File: lab6_bai3.py

Chuẩn bị input MST

```
vertices = ['A', 'B', 'C', 'D', 'E']
edges = [
    (1, 'A', 'B'),
    (4, 'A', 'C'),
    (3, 'B', 'C'),
    (2, 'B', 'D'),
    (5, 'C', 'E'),
    (2, 'D', 'E'),
]
# Mỗi cạnh: (weight, u, v)
```

Yêu cầu SV: vẽ đồ thị này ra giấy.

Code Kruskal dùng DSU

```
from lab6_bai2 import make_set, find, union

def kruskal_mst_basic(vertices, edges):
    """
    Kruskal MST dùng DSU basic (chưa tối ưu).
    """
    # B1: sort cạnh theo trọng số
    edges_sorted = sorted(edges, key=lambda e: e[0])

    # B2: khởi tạo DSU
    parent = make_set(vertices)
```

```

mst = []
total_weight = 0

print("Cạnh sau khi sort (w, u, v):")
for e in edges_sorted:
    print(" ", e)

print("\nDuyệt từng cạnh:")
for w, u, v in edges_sorted:
    root_u = find(parent, u)
    root_v = find(parent, v)
    print(f"Xét cạnh {u}-{v} (w={w}), root_u={root_u}, root_v={root_v}")

    if root_u != root_v:
        print(" → Khác nhóm → CHỌN cạnh này")
        mst.append((u, v, w))
        total_weight += w
        union(parent, u, v)
    else:
        print(" → Cùng nhóm → BỎ (tránh chu trình)")

# Nếu đã đủ |V|-1 cạnh, có thể dừng luôn
if len(mst) == len(vertices) - 1:
    break

return mst, total_weight

```

Sinh viên:

- Chạy, quan sát log.

- So sánh với việc làm Kruskal bằng tay.

Kruskal dùng DSU optimized

```
from lab6_bai2 import make_set_optimized, find_optimized, union_optimized

def kruskal_mst_optimized(vertices, edges):
    """
    Kruskal MST với DSU tối ưu: path compression + union by size.
    """
    edges_sorted = sorted(edges, key=lambda e: e[0])
    parent, size = make_set_optimized(vertices)

    mst = []
    total_weight = 0

    for w, u, v in edges_sorted:
        if find_optimized(parent, u) != find_optimized(parent, v):
            mst.append((u, v, w))
            total_weight += w
            union_optimized(parent, size, u, v)
        if len(mst) == len(vertices) - 1:
            break

    return mst, total_weight
```

Yêu cầu SV:

- So sánh output của `kruskal_mst_basic` và `kruskal_mst_optimized` – phải giống nhau.
- Giải thích: optimized chỉ khác ở TỐC ĐỘ khi đồ thị lớn.

Hàm test tổng hợp MST

```
def test_kruskal():  
    vertices = ['A', 'B', 'C', 'D', 'E']  
    edges = [  
        (1, 'A', 'B'),  
        (4, 'A', 'C'),  
        (3, 'B', 'C'),  
        (2, 'B', 'D'),  
        (5, 'C', 'E'),  
        (2, 'D', 'E'),  
    ]  
  
    print("=== Kruskal với DSU basic ===")  
    mst1, total1 = kruskal_mst_basic(vertices, edges)  
    print("\nMST basic:")  
    for u, v, w in mst1:  
        print(f" {u}-{v} (w={w})")  
    print("Tổng trọng số:", total1)  
  
    print("\n=== Kruskal với DSU optimized ===")  
    mst2, total2 = kruskal_mst_optimized(vertices, edges)  
    print("\nMST optimized:")  
    for u, v, w in mst2:  
        print(f" {u}-{v} (w={w})")  
    print("Tổng trọng số:", total2)  
  
if __name__ == "__main__":  
    test_kruskal()
```